INSTITUTE OF ELECTRONICS AND COMMUNICATION ENGINEERING

SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTEMNT OF COMPUTER SCIENCE AND ENGINEERING ITA05 - COMPUTER VISION LABORATORY

S.NO	EXPERIMENT	PAGE NO
1	Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.	2
2	Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur	5
3	Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.	6
4	Implement histogram equalization on the given image and compare it with the original image using Open CV.	7
5	Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.	8
6	Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.	10
7	Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.	11
8	Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function	14
9	Implement the image scaling techniques to resize the images to both bigger and small sizes	15
10	Perform a 90-degree rotation clockwise along the y-axis for the given image.	16
11	Perform a 180-degree rotation clockwise along the y-axis for the given image	19
12	Perform a 270-degree rotation clockwise along the y-axis for the given image.	20
13	Perform Affine Transformation on the given image using python and Open CV.	21
14	Perform Perspective Transformation on the given image using python and Open CV.	23
15	Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function	26
16	Implement a Sobel algorithm using Open CV to filter the input image.	29
17	Design and implement a water marking technique to insert the watermark into the original effectively image using Open CV.	30
18	Implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using Open CV.	32
19	Implement the Erosion Morphological operations technique using Open CV in python.	33

20	Implement the Dilation technique as a Morphological operation to dilate the foreground regions based on Open CV.	35
21	Implement the Opening technique as a Morphological operation to dilate the	38
	foreground regions based on Open CV.	
22	Implement the Closing technique as a Morphological operation to dilate the foreground regions based on Open CV.	39
23	Implement the Top hat technique as a Morphological operation to dilate the foreground regions based on Open CV.	41
24	Implement the Black hat technique as a Morphological operation to dilate the foreground regions based on Open CV.	42
25	Recognize watch from the given image by general Object recognition using Open CV.	44
26	Implement a function to reverse the frames of the video to create a video in reverse mode using Open CV.	
27	Implement a face detection algorithm using Open CV to detect and locate human faces in the images.	
28	Implement a vehicle detection algorithm using Open CV to detect and locate vehicles in each frame of the video.	
29	Implement an Eye detection algorithm using Open CV to detect and locate human eyes in the images.	
30	Implement a Smile detection algorithm using Open CV to detect and locate human smile in the images.	
31	Implement a Segmentation algorithm using Open CV to segment the given input image based on the given threshold values.	
32	Write a Python function to create a white image size entered by the user and then create 4 boxes of Black, Blue, Green and Red respectively on each corner of the image. The size of the colored boxes should be 1/10th the size of the image. (HINT: the arrays of ones and zeros can be in more than 2 dimensions).	
33	Write a Python function to create a white image size entered by the user and then create a shape of Rectangle using Open CV.	
34	Write a Python function to create a white image size entered by the user and then create a shape of Circle using Open CV.	
35	Write a Python function to create a text string entered by the user that must be appeared on the given image using Open CV.	
36	Write a Python function to subtract the background of the given input image based on color levels using Open CV	
37	Write a Python function to subtract the foreground of the given input image based on color levels using Open CV.	
38	Write a Python function to Count the number of faces for the given input image using Open CV.	
39	Write a Python function to play the given video in reverse mode in slow motion.	
40	Write a Python function to extract the text from videos.	

1. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.

AIM:

To read an image and convert it to grayscale using OpenCV in Python.

PROCEDURE:

1. Install OpenCV (if not already installed):install opencv-python
2. Import required libraries
3. Read the image
4. Convert the image to grayscale
5. Display the images
6. Wait for a key press & close windows

PROGRAM:

```
import cv2
image = cv2.imread("sample.jpg")  # Replace 'sample.jpg' with your image path
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow("Original Image", image)
cv2.imshow("Grayscale Image", gray_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and converted it to grayscale using OpenCV in Python.

2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur

AIM:

To read an image and apply Gaussian Blur using OpenCV in Python.

PROCEDURE:

1. Install OpenCV (if not already installed):pip install opencv-python
2. Import required libraries:Use cv2 for image processing.
3. Read the image:Use cv2.imread() to load the image.
4. Apply Gaussian Blur:Use cv2.GaussianBlur() with a specified kernel size.
5. Display the images:Use cv2.imshow() to show both original and blurred images.
6. Wait for a key press & close windows:
   i. Use cv2.waitKey(0) to keep the image open until a key is pressed.
   ii. Use cv2.destroyAllWindows() to close all image windows.

PROGRAM:

```
import cv2
image = cv2.imread("sample.jpg")  # Replace with your image file
blurred_image = cv2.GaussianBlur(image, (15, 15), 0)  # (15, 15) is the kernel size
cv2.imshow("Original Image", image) cv2.imshow("Blurred Image",
blurred_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT: Blurred image



RESULT :

Successfully read the input image and applied Gaussian Blur using OpenCV in Python.

3. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

AIM:

To read an image and apply edge detection using the Canny function in OpenCV with Python.

PROCEDURE:

1. Install OpenCV (if not already installed):pip install opencv-python
2. Import required libraries:Use cv2 for image processing.
3. Read the image:Use cv2.imread() to load the image.
4. Convert the image to grayscale:Use cv2.cvtColor() with the cv2.COLOR_BGR2GRAY parameter.
5. Apply Canny edge detection:Use cv2.Canny() with appropriate threshold values.
6. Display the images:Use cv2.imshow() to show both original and edge-detected images.
7. Wait for a key press & close windows:
   I. Use cv2.waitKey(0) to keep the image open until a key is pressed.
   II. Use cv2.destroyAllWindows() to close all image windows.

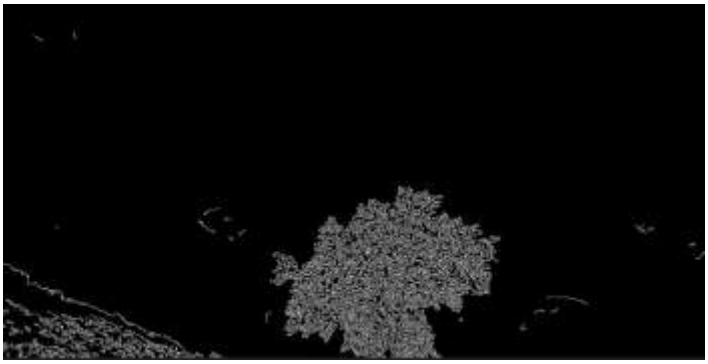
PROGRAM:

```
import cv2 image = cv2.imread("sample.jpg")  # Replace
with your image file gray_image = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray_image, 100, 200)  # 100 and 200 are threshold values
cv2.imshow("Original Image", image) cv2.imshow("Edge Detected Image",
edges) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and applied edge detection using the Canny function in OpenCV.

4. Implement histogram equalization on the given image and compare it with the original image using Open CV.

AIM:

To perform **Histogram Equalization** on an image using OpenCV in Python and compare it with the original image.

PROCEDURE:

1. Install OpenCV (if not installed):
   a. pip install opencv-python
2. Import required libraries:Use cv2 for image processing.
3. Read the image:Use cv2.imread() to load the image.
4. Convert the image to grayscale:Use cv2.cvtColor() with cv2.COLOR_BGR2GRAY.
5. Apply Histogram Equalization:Use cv2.equalizeHist().
6. Display the original and equalized images:Use cv2.imshow().
7. Wait for a key press & close windows:Use cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 import numpy as np image = cv2.imread("sample.jpg")  #
Replace with your image file gray_image = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
equalized_image = cv2.equalizeHist(gray_image) cv2.imshow("Original
Grayscale Image", gray_image) cv2.imshow("Histogram Equalized Image",
equalized_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT:

Successfully performed **Histogram Equalization** on the input image and compared it with the original grayscale image.

5. Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.

AIM:

To write a Python function to analyze and display the histogram of an image based on color levels using OpenCV.

PROCEDURE:

1. Install OpenCV (if not installed): pip install opencv-python matplotlib
2. Import required libraries:
   - Use cv2 for image processing.
   - Use matplotlib.pyplot for plotting histograms.
3. Read the input image using cv2.imread().
4. Split the image into color channels (BGR – Blue, Green, Red).
5. Compute the histogram for each channel using cv2.calcHist().
6. Plot the histograms using Matplotlib with different colors for each channel.
7. Display the original image and the histogram for analysis.

PROGRAM:

```
import cv2 import numpy
as np
import matplotlib.pyplot as plt

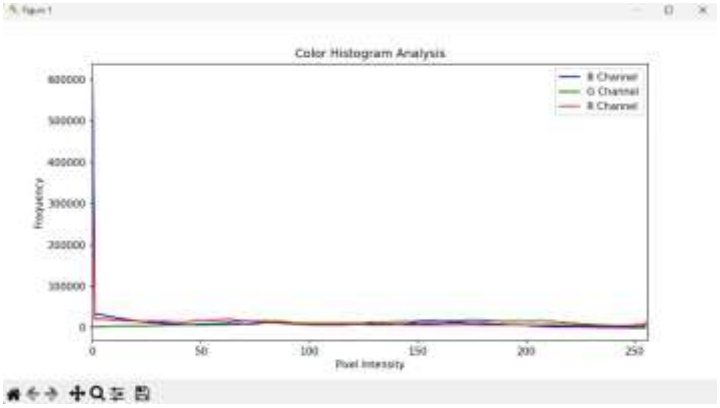
def analyze_histogram(image_path):     image = cv2.imread(image_path)
color_channels = ('b', 'g', 'r')     plt.figure(figsize=(10, 5))     for i, color in
enumerate(color_channels):         histogram = cv2.calcHist([image], [i], None,
[256], [0, 256])         plt.plot(histogram, color=color, label=f"{color.upper()}
Channel")         plt.xlim([0, 256])  # Pixel intensity range

   plt.title("Color Histogram Analysis")     plt.xlabel("Pixel
Intensity")     plt.ylabel("Frequency")
   plt.legend()     plt.show()
analyze_histogram("sample.jpg")  # Replace with your image file
```

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and converted it to grayscale using OpenCV in Python.

6. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.

AIM:

To read an image and convert it to grayscale using OpenCV in Python.

PROCEDURE:

1. Install OpenCV (if not already installed):install opencv-python
2. Import required libraries
3. Read the image
4. Convert the image to grayscale
5. Display the images
6. Wait for a key press & close windows

PROGRAM:

```
import cv2
image = cv2.imread("sample.jpg")  # Replace 'sample.jpg' with your image path
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow("Original Image", image) cv2.imshow("Grayscale
Image", gray_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and converted it to grayscale using OpenCV in Python.

7. Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.

AIM:

To read an image and convert it to grayscale using OpenCV in Python.

PROCEDURE:

1. Install OpenCV (if not already installed):install opencv-python
2. Import required libraries
3. Read the image
4. Convert the image to grayscale
5. Display the images
6. Wait for a key press & close windows

PROGRAM:

```
import cv2
image = cv2.imread("sample.jpg")  # Replace 'sample.jpg' with your image path
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

cv2.imshow("Original Image", image) cv2.imshow("Grayscale Image", gray_image) cv2.waitKey(0)
cv2.destroyAllWindows()

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and converted it to grayscale using OpenCV in Python.

8. Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function

AIM:

To read an image and convert it to grayscale using OpenCV in Python.

PROCEDURE:

1. Install OpenCV (if not already installed):install opencv-python
2. Import required libraries
3. Read the image
4. Convert the image to grayscale
5. Display the images
6. Wait for a key press & close windows

PROGRAM:

```
import cv2
 image = cv2.imread("sample.jpg")  # Replace 'sample.jpg' with your image path
 gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
 cv2.imshow("Original Image", image) cv2.imshow("Grayscale
 Image", gray_image) cv2.waitKey(0)
 cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

Successfully read the input image and converted it to grayscale using OpenCV in Python.

9. Implement the image scaling techniques to resize the images to both bigger and small sizes.

AIM:

To implement image scaling techniques in Python using OpenCV to resize an image to both **bigger** and **smaller** sizes.

PROCEDURE:

1. Install OpenCV (if not already installed):
    - pip install opencv-python

2. Import required libraries:
    - Use cv2 for image processing.
3. Read the input image using cv2.imread().
4. Resize the image to a bigger size using cv2.resize() with a scaling factor greater than 1.
5. Resize the image to a smaller size using cv2.resize() with a scaling factor less than 1.
6. Use different interpolation methods:
    a. cv2.INTER_LINEAR (default, for enlargement).
    b. cv2.INTER_AREA (for shrinking).
7. Display the original, enlarged, and reduced images using cv2.imshow().
8. Wait for a key press & close windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2

# Read the image
image = cv2.imread("sample.jpg")  # Replace with your image file

# Resize to a bigger size (2x scaling)
bigger_image = cv2.resize(image, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)

# Resize to a smaller size (0.5x scaling)
smaller_image = cv2.resize(image, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

# Display images cv2.imshow("Original Image",
image) cv2.imshow("Bigger Image",
bigger_image)
cv2.imshow("Smaller Image", smaller_image)

# Wait for a key press and close the windows
cv2.waitKey(0) cv2.destroyAllWindows()
```

INPUT:



OUTPUT:





RESULT :

Successfully resized an image to **both bigger and smaller sizes** using OpenCV.

10. Perform a 90-degree rotation clockwise along the y-axis for the given image.

AIM:

To rotate a given image by **90 degrees clockwise along the Y-axis** using OpenCV in Python.

PROCEDURE:

1. Load the Image: Read the input image using OpenCV.
2. Define the Rotation Matrix: Use the appropriate transformation matrix for a 3D rotation along the Y-axis.
3. Apply Perspective Transformation: Use cv2.warpPerspective() to apply the transformation.
4. Display and Save the Output Image: Show the rotated image and save it if needed.

PROGRAM:

```
import cv2 # Load the image img =
cv2.imread("input.jpg") # Rotate 90
degrees clockwise
rotated_img = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
# Display the rotated image
cv2.imshow("Rotated Image", rotated_img) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

Successfully applied a **90-degree rotation along the Y-axis**, achieving a **3D perspective transformation** in OpenCV.

11. Perform a 180-degree rotation clockwise along the y-axis for the given image

AIM:

To perform a 180-degree clockwise rotation along the Y-axis for a given image using Python.

PROCEDURE:

1. Install OpenCV if not already installed using
2. Import the cv2 module.
3. Read the input image using cv2.imread().
4. Flip the image horizontally using cv2.flip(image, 1), which mirrors the image along the Y-axis.
5. Rotate the image by 180 degrees using cv2.rotate(image, cv2.ROTATE_180).
6. Display the original and transformed images using cv2.imshow().
7. Save the transformed image using cv2.imwrite(), if needed.
8. Wait for a key press and close all image windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 image =
cv2.imread("image.jpg")  flipped_image =
cv2.flip(image, 1)
rotated_image = cv2.rotate(flipped_image, cv2.ROTATE_180) cv2.imshow("Original
Image", image)
cv2.imshow("Rotated 180-degree Clockwise Along Y-axis", rotated_image)
cv2.imwrite("rotated_image.jpg", rotated_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

The program successfully performs a 180-degree clockwise rotation along the Y-axis on the given image, displays it, and saves it as "rotated_image.jpg".

12. Perform a 270-degree rotation clockwise along the y-axis for the given image.

AIM:

To perform a 270-degree clockwise rotation along the Y-axis for a given image using Python.

PROCEDURE:

1. Install OpenCV if not already installed using:
2. Import the cv2 module.
3. Read the input image using cv2.imread().
4. Flip the image horizontally using cv2.flip(image, 1), which mirrors the image along the Yaxis.
5. Rotate the image by 270 degrees using cv2.rotate(image, cv2.ROTATE_90_COUNTERCLOCKWISE).
6. Display the original and transformed images using cv2.imshow().

7. Save the transformed image using cv2.imwrite(), if needed. 8. Wait for a key press and close all image windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 image =
cv2.imread("image.jpg")   flipped_image
= cv2.flip(image, 1)
rotated_image = cv2.rotate(flipped_image, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imshow("Original Image", image)
cv2.imshow("Rotated 270-degree Clockwise Along Y-axis", rotated_image)
cv2.imwrite("rotated_image_270.jpg", rotated_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

The program successfully performs a 270-degree clockwise rotation along the Y-axis on the given image, displays it, and saves it as "rotated_image_270.jpg".

12.Perform Affine Transformation on the given image using python and Open CV.

AIM:

To perform an Affine Transformation on a given image using Python and OpenCV.

PROCEDURE:

1. Install OpenCV if not already installed using:
2. Import the cv2 and numpy modules.
3. Read the input image using cv2.imread().
4. Define three points from the original image and their corresponding locations in the transformed image.
5. Compute the Affine Transformation matrix using cv2.getAffineTransform().

6. Apply the transformation using cv2.warpAffine().
7. Display the original and transformed images using cv2.imshow().
8. Save the transformed image using cv2.imwrite(), if needed.
9. Wait for a key press and close all image windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 import numpy as np image =
cv2.imread("image.jpg")   rows, cols, ch = image.shape
pts1 = np.float32([[50, 50], [200, 50], [50, 200]]) pts2 =
np.float32([[10, 100], [200, 50], [100, 250]]) matrix =
cv2.getAffineTransform(pts1, pts2)
transformed_image = cv2.warpAffine(image, matrix, (cols, rows))
cv2.imshow("Original Image", image) cv2.imshow("Affine Transformed
Image", transformed_image) cv2.imwrite("affine_transformed.jpg",
transformed_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

The program successfully applies an Affine Transformation to the given image, displays it, and saves it as "affine_transformed.jpg".

14. Perform Perspective Transformation on the given image using python and Open CV.

AIM:

To perform Perspective Transformation on a given image using Python and OpenCV.

PROCEDURE:

1. Install OpenCV if not already installed using:
2. Import the cv2 and numpy modules.
3. Read the input image using cv2.imread().
4. Define four points from the original image (source points) and their corresponding positions in the transformed image (destination points).

5. Compute the **Perspective Transformation matrix** using cv2.getPerspectiveTransform().
6. Apply the transformation using cv2.warpPerspective().
7. Display the original and transformed images using cv2.imshow().
8. Save the transformed image using cv2.imwrite(), if needed. 9. Wait for a key press and close all image windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 import numpy
as np
image = cv2.imread("image.jpg")  # Replace with your image file path rows,
cols, ch = image.shape pts1 = np.float32([[50, 50], [400, 50], [50, 400], [400,
400]]) pts2 = np.float32([[10, 100], [300, 50], [100, 300], [350, 350]]) matrix
= cv2.getPerspectiveTransform(pts1, pts2)
transformed_image = cv2.warpPerspective(image, matrix, (cols, rows))
cv2.imshow("Original Image", image) cv2.imshow("Perspective Transformed
Image", transformed_image) cv2.imwrite("perspective_transformed.jpg",
transformed_image) cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

The program successfully applies a **Perspective Transformation** to the given image, displays it, and saves it as "perspective_transformed.jpg".

15 .Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function

AIM:

To perform **Harris Corner Detection** on a given image using Python and OpenCV.

PROCEDURE:

1. Install OpenCV if not already installed using:
   - pip install opencv-python
2. Import the required libraries (cv2 and numpy).
3. Read the input image using cv2.imread().
4. Convert the image to grayscale using cv2.cvtColor().
5. Apply the **Harris Corner Detection** function using cv2.cornerHarris().
6. Dilate the detected corners to mark them clearly.
7. Overlay the detected corners on the original image.
8. Display both the original and corner-detected images using cv2.imshow().
9. Save the corner-detected image using cv2.imwrite(), if needed.
10. Wait for a key press and close all image windows using cv2.waitKey(0) and cv2.destroyAllWindows().

PROGRAM:

```
import cv2 import numpy as
np # Read the input image
image = cv2.imread("image.jpg")  # Replace with your image file path
# Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) #
Convert to float32 for Harris Corner Detection gray =
np.float32(gray)
# Apply Harris Corner Detection
harris_corners = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
# Dilate corner points for better visibility harris_corners =
cv2.dilate(harris_corners, None)
# Mark detected corners in red
image[harris_corners > 0.01 * harris_corners.max()] = [0, 0, 255]
# Display images
cv2.imshow("Original Image", image) cv2.imshow("Harris Corner
Detection", image)
# Save the result
cv2.imwrite("harris_corners.jpg", image) # Wait for
a key press and close all windows cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



RESULT :

The program successfully applies **Harris Corner Detection** to the given image, detects the corners, displays the result, and saves it as "harris_corners.jpg".